

Competitive Programming

Section 47 Editorial: Meet-In-The-Middle

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Count Subset Sums $\leq X$	4
2.1	Restatement	4
2.2	Algorithm and proof	4
2.3	Complexity	4
2.4	Full C++ solution	4
2.5	Full Python solution	4
3	B. Best Subset Sum $\leq X$	5
3.1	Restatement	5
3.2	Algorithm and proof	5
3.3	Complexity	5
3.4	Full C++ solution	5
3.5	Full Python solution	5
4	C. Closest Subset Sum	6
4.1	Restatement	6
4.2	Algorithm and proof	6
4.3	Complexity	6
4.4	Full C++ solution	6
4.5	Full Python solution	6
5	Testing discussion	7

1 Editorial overview

1.1 Problem-specific proof map

A. Count Subset Sums $\leq X$. Every subset is uniquely a left subset plus a right subset. Sorting right sums lets each left sum count compatible partners with `upper_bound`.

B. Best Subset Sum $\leq X$. The same unique split applies; binary-search the greatest right sum no larger than `X-left` and maximize over left sums.

C. Closest Subset Sum. The closest right partner to `target-left` is one of the two sorted neighbors around a lower bound, so checking both covers every optimal full subset.

1.2 Detailed solution guide

A. Count Subsets with Sum at Most S . Split the array into left and right halves. Enumerate every subset sum of each half. For a fixed left sum x , the valid right sums are $y \leq S - x$; after sorting the right sums, their count is `upper_bound(right, S-x)`. Every original subset has exactly one left part and one right part, so the sum over all left sums counts each valid subset once.

B. Maximum Subset Sum at Most S . Use the same split and sorted right sums. For each left sum $x \leq S$, find the largest right sum $y \leq S-x$ and maximize $x+y$. This is the optimization version of A: instead of counting all compatible partners, keep the best compatible partner.

C. Closest Subset Sum. Enumerate both halves. For each left sum x , the best right partner is near `target - x` in sorted order. Check `lower_bound` and its predecessor, because one is the first sum not below the target gap and the other is the largest sum below it. One of those two minimizes the absolute difference for this x ; scanning all x gives the global optimum.

Common pitfalls. Include the empty subset in both halves. Use 64-bit integers for sums. The enumeration cost is $2^{(n/2)}$, which is the whole point of the technique.

Meet-in-the-middle splits one exponential choice into two half-sized enumerations.

- A counts pairs of half-subset sums whose total is at most X .
- B maximizes such a total instead of counting it.
- C checks the two right-half sums nearest to `T-left_sum`.

Every full subset is uniquely determined by its left half and right half, so enumerating both lists is complete. Sorting one side avoids trying all pairs.

2 A. Count Subset Sums $\leq X$

2.1 Restatement

Count subsets whose sum is at most X .

2.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. `\square`

2.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<long long>sums(vector<long long>a){vector<long long>r{0};for(lon
```

2.5 Full Python solution

```
import sys,bisect
def sums(a):
    res=[0]
    for x in a: res += [y+x for y in res]
    return res
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,x=data[0],data[1]; a=data[2:]; left=sums(a[:n//2]); right=sorted(sums(a[n//2:])); ans=
    for s in left: ans+=bisect.bisect_right(right,x-s)
    print(ans)
if __name__=="__main__": main()
```

3 B. Best Subset Sum $\leq X$

3.1 Restatement

Print the maximum subset sum not exceeding X .

3.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. \square

3.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<long long>sums(vector<long long>a){vector<long long>r{0};for(lon
```

3.5 Full Python solution

```
import sys,bisect
def sums(a):
    res=[0]
    for x in a: res += [y+x for y in res]
    return res
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,x=data[0],data[1]; a=data[2:]; left=sums(a[:n//2]); right=sorted(sums(a[n//2:])); best=0
    for s in left:
        i=bisect.bisect_right(right,x-s)-1
        if i>=0: best=max(best,s+right[i])
    print(best)
if __name__=="__main__": main()
```

4 C. Closest Subset Sum

4.1 Restatement

Print the minimum absolute difference between a subset sum and target T.

4.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. $\square$

4.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<long long>sums(vector<long long>a){vector<long long>r{0};for(lon
```

4.5 Full Python solution

```
import sys,bisect
def sums(a):
    res=[0]
    for x in a: res += [y+x for y in res]
    return res
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,t=data[0],data[1]; a=data[2:]; L=sums(a[:n//2]); R=sorted(sums(a[n//2:])); best=10**30
    for s in L:
        i=bisect.bisect_left(R,t-s)
        for j in (i-1,i):
            if 0<=j<len(R): best=min(best,abs(s+R[j]-t))
    print(best)
if __name__=="__main__": main()
```

5 Testing discussion

The proof is a bijection between full subsets and pairs of half-subsets. For small n , enumerate all masks as the stress oracle. In counting, use `upper_bound(X-left_sum)`; in optimization, verify the candidate immediately below that bound, never merely the first greater value.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.